



Proyecto 2: Simulador P2P de Archivos

PRINCIPIOS DE SISTEMAS OPERATIVOS

Estudiantes:

Manfred Azofeifa Salazar
David Alonso Zamora Barrantes
Diego Zamora Bustos

Profesor:

Eddy Ramírez Jiménez

21 de junio de 2026

Índice

1. Introducción	2
1.1. Objetivos del proyecto	2
1.2. Motivación y características del sistema	2
1.3. Componentes principales	2
2. Marco Teórico	4
3. Descripción de la Solución	5
3.1. Arquitectura General	5
3.2. Servidor Central	5
3.3. Cliente	6
3.4. Función de Hash Propia	6
3.5. Transferencia de Archivos	6
3.6. Búsqueda Distribuida	7
3.7. Concurrencia y Sincronización	7
3.8. Interfaz de Consola	8
4. Descripción de la Experiencia	8
4.1. Problemas Encontrados	8
4.2. Decisiones Únicas	9
5. Aprendizajes Personales	9
5.1. Aprendizaje de Manfred	9
5.2. Aprendizaje de David	9
5.3. Aprendizaje de Diego	10
6. Conclusiones	10

1. Introducción

1.1. Objetivos del proyecto

El presente proyecto tiene como finalidad afianzar los conceptos fundamentales de sistemas operativos mediante la implementación de un sistema de intercambio de archivos entre pares (P2P). Los objetivos formativos que respaldan el trabajo son:

1. Proporcionar una comprensión sólida de la teoría y práctica básica de los Sistemas Operativos.
2. Valorar implementaciones modernas de Sistemas Operativos.
3. Dar un criterio con respecto a las diferentes soluciones que se pueden dar ante los problemas de los Sistemas Operativos.
4. Utilizar los criterios aprendidos para encontrar las mejores soluciones a problemas similares de programación.

1.2. Motivación y características del sistema

El propósito del proyecto es familiarizar al estudiante con problemas relativos a sistemas operativos, tales como calidad de servicio, manejo de E/S, comunicación entre procesos mediante sockets, búsqueda en sistemas de archivos, hashing y búsqueda distribuida.

El sistema implementa una red P2P donde los clientes comparten archivos a través de una carpeta común. Cada cliente registra sus archivos en un servidor central, que mantiene metadatos (hash, tamaño y dueño). Los clientes pueden buscar archivos por nombre (de forma centralizada o distribuida) y descargarlos desde uno o varios pares simultáneamente, utilizando un protocolo de transferencia por rangos. La búsqueda distribuida emplea un mecanismo de inundación controlada (flooding) con TTL y caché de consultas vistas para evitar ciclos.

1.3. Componentes principales

- **Servidor central:** almacena metadatos de los archivos registrados, atiende solicitudes de registro, búsqueda por nombre (**FIND**) y búsqueda por hash/tamaño (**LOOKUP**). Además, proporciona una lista de vecinos a cada cliente al registrarse.
- **Cliente P2P:** al iniciar, escanea su carpeta compartida, calcula hash de cada archivo y se registra en el servidor. Ofrece una consola interactiva con comandos para buscar y descargar archivos. También actúa como servidor de transferencia, atendiendo peticiones de otros clientes.
- **Protocolo de comunicación:** basado en texto ASCII con líneas terminadas en `\n`. Los mensajes son simples y están definidos en el archivo `protocol.h`.

- **Búsqueda distribuida:** los clientes mantienen una lista de vecinos obtenida del servidor. Cuando la búsqueda centralizada falla o no se obtienen resultados, se propaga una consulta a través de la red de vecinos con TTL y control de duplicados.
- **Transferencia segmentada:** los archivos se descargan en rangos desde múltiples pares, permitiendo descargas paralelas y reensamblado en el cliente.

2. Marco Teórico

Sockets y Comunicación en Red	Protocolos de Capa de Aplicación
Los sockets TCP permiten la comunicación confiable entre procesos ubicados en diferentes equipos. En este proyecto se utilizan para registrar archivos, realizar búsquedas y transferir datos entre clientes y servidor.	El sistema emplea un protocolo textual basado en comandos y respuestas. Este enfoque simplifica la implementación, la depuración y la ampliación futura del sistema.
Hilos y Concurrencia	Sincronización con Mutex y Condiciones
Los hilos permiten ejecutar múltiples tareas de manera simultánea, como atender conexiones, procesar búsquedas y realizar transferencias sin bloquear otras operaciones.	Los mutex protegen estructuras de datos compartidas, mientras que las variables de condición permiten coordinar la ejecución de los hilos cuando deben esperar eventos específicos.
Función de Hash Propia	Búsqueda Distribuida (Flooding)
La función hash genera un identificador único para cada archivo según su contenido. Esto permite localizar archivos y verificar su integridad después de una descarga.	La búsqueda distribuida propaga consultas entre los vecinos de la red para localizar archivos que no se encuentran mediante la búsqueda centralizada.
TTL y Control de Propagación	Transferencia por Rangos y Reensamblado
El TTL limita la cantidad de saltos que puede realizar una consulta dentro de la red, reduciendo el tráfico innecesario y evitando ciclos.	Los archivos pueden descargarse desde varios pares de manera simultánea mediante segmentos. Posteriormente, los fragmentos se ensamblan para reconstruir el archivo completo.

3. Descripción de la Solución

3.1. Arquitectura General

El sistema se compone de un servidor central y múltiples clientes. La comunicación se realiza mediante sockets TCP. El servidor mantiene una tabla de metadatos de archivos registrados y una lista de clientes recientes. Los clientes se registran, publican sus archivos, y luego pueden buscar y descargar archivos de otros clientes.

La estructura del código está organizada en los siguientes módulos principales:

- **server.c**: Implementa el servidor central. Maneja conexiones entrantes, procesa comandos (**REGISTER**, **FIND**, **LOOKUP**) y gestiona las estructuras de metadatos.
- **client.c**: Punto de entrada del cliente. Procesa argumentos, escanea la carpeta compartida, se registra en el servidor y lanza el servidor de transferencia.
- **console.c**: Implementa la interfaz de consola del cliente, procesando comandos como **find** y **request**.
- **transfer.c**: Contiene la lógica de transferencia de archivos: manejo de peticiones **GET**, envío de rangos, descarga desde uno o varios pares, y reensamblado.
- **discovery.c**: Implementa el protocolo de búsqueda distribuida: manejo de mensajes **DSEARCH** y **DRESULT**, control de TTL, caché de consultas vistas.
- **hash.c**: Función de hash propia para calcular el hash de archivos.
- **protocol.h**: Define constantes y estructuras compartidas para el protocolo.

3.2. Servidor Central

El servidor se ejecuta en un bucle infinito aceptando conexiones. Por cada conexión, crea un hilo que lee la primera línea y determina el comando. Los comandos soportados son:

- **REGISTER <puerto_transferencia><cantidad_archivos>**: Inicia el registro. El cliente envía luego una línea **FILE** por cada archivo y termina con **END**. El servidor almacena los metadatos en arrays globales protegidos por un mutex.
- **FIND <nombre>**: Busca archivos cuyo nombre coincida exactamente con el término. Devuelve una lista de pares (IP:puerto) que tienen ese archivo.
- **LOOKUP <tamaño><hash>**: Busca archivos por tamaño y hash exactos. Devuelve la lista de pares que poseen ese archivo.

Al completar un registro exitoso, el servidor responde con `OK` y una lista de `NEIGHBORS` que contiene los clientes más recientes (hasta 4). Esta lista permite a los clientes iniciar búsquedas distribuidas.

3.3. Cliente

El cliente, al iniciar, realiza los siguientes pasos:

1. Lee los argumentos: IP del servidor, puerto del servidor, puerto de transferencia y carpeta compartida.
2. Escanea la carpeta compartida con `revisar_carpeta_compartida()`. Por cada archivo regular, calcula su hash y tamaño, y lo almacena en una estructura local.
3. Se conecta al servidor y envía el registro con todos los archivos. Recibe la lista de vecinos y la almacena.
4. Inicia un servidor de transferencia en el puerto especificado, que escucha peticiones `GET` de otros clientes.
5. Entra en el bucle de consola, esperando comandos del usuario.

El servidor de transferencia, al recibir una conexión, crea un hilo que lee el comando `GET`. Si el archivo solicitado existe localmente (por hash y tamaño), envía el rango solicitado con `DATA`. Además, el mismo hilo atiende mensajes `DSEARCH` y `DRESULT` para la búsqueda distribuida.

3.4. Función de Hash Propia

La función de hash se implementa en `hash.c` y utiliza dos registros de 64 bits para procesar el contenido de cada archivo. Mediante operaciones de mezcla y rotación de bits genera un identificador hexadecimal de 128 bits. Este identificador se utiliza para localizar archivos y verificar su integridad durante las transferencias.

3.5. Transferencia de Archivos

La transferencia se realiza mediante el protocolo `GET/DATA`. El cliente que desea descargar un archivo:

1. Consulta al servidor con `LOOKUP <tamaño><hash>` para obtener la lista de pares.
2. Si la lista contiene un solo par, inicia una descarga completa usando `descargar_de_par()`.
3. Si contiene varios pares, divide el archivo en rangos iguales (tantos como pares) y lanza un hilo por rango con `descargar_de_pares()`.

El hilo escribe los bytes en la posición adecuada de un archivo temporal (`.part`). Si algún rango falla, se reintenta con otro par de la lista. Una vez completados todos los rangos, se verifica el hash del archivo temporal contra el esperado y, si coincide, se renombra a su nombre definitivo (el hash). El archivo se guarda en la carpeta compartida del cliente solicitante.

3.6. Búsqueda Distribuida

La búsqueda distribuida se activa con el comando `find -d <nombre>` o automáticamente cuando `find` no obtiene resultados del servidor.

El proceso es el siguiente:

1. El originador genera un `query_id` único combinando timestamp, PID y puerto.
2. Construye un mensaje `DSEARCH <query_id><ip_origen><puerto_origen><ttl><termino>`.
3. Envía el mensaje a todos sus vecinos (almacenados de la lista obtenida del servidor).
4. Cada vecino que recibe el mensaje:
 - Verifica si ya procesó ese `query_id` (caché de consultas vistas). Si es así, descarta el mensaje.
 - Si no, lo registra en el caché con el timestamp actual.
 - Busca en sus archivos locales si alguno coincide con el término (comparación exacta de nombre).
 - Si encuentra coincidencia, envía un `DRESULT` directamente al originador, con los metadatos del archivo y su propia IP:puerto.
 - Si el TTL ≥ 1 , decrementa el TTL y reenvía el mensaje a sus vecinos (excepto al que se lo envió).
5. El originador espera 500 ms recolectando respuestas, luego las presenta al usuario.

El caché de consultas vistas expira después de 120 segundos, permitiendo que las consultas antiguas sean procesadas nuevamente si es necesario.

3.7. Concurrencia y Sincronización

El servidor central protege sus estructuras globales con un mutex (`candado_registro`). Cada hilo de cliente adquiere este mutex al modificar las tablas de metadatos y clientes recientes.

En el cliente, la búsqueda distribuida utiliza un mutex y una variable de condición para esperar los resultados. El estado de la consulta activa se guarda en

una estructura global (`estado_consulta_distribuida_t`). Los hilos que reciben DRESULT señalizan la variable de condición para despertar al hilo principal.

El servidor de transferencia del cliente crea un hilo por cada conexión entrante, por lo que no hay bloqueos. Cada hilo maneja su propia sesión y libera los recursos al finalizar.

3.8. Interfaz de Consola

La consola del cliente acepta los siguientes comandos:

- **archivos**: Muestra los archivos locales registrados (tamaño, hash, nombre).
- **vecinos**: Muestra la lista de vecinos conocidos.
- **find -s <nombre>**: Búsqueda centralizada.
- **find -d <nombre>**: Búsqueda distribuida.
- **find <nombre>**: Primero intenta centralizada; si falla, distribuirá automáticamente.
- **request <tamaño><hash>**: Descarga el archivo especificado.
- **quit** o **exit**: Termina el cliente.

El manejo de la entrada es simple y se realiza con `fgets()`. Los comandos se procesan en el hilo principal, sin bloquear las transferencias en curso.

4. Descripción de la Experiencia

4.1. Problemas Encontrados

Uno de los primeros desafíos fue diseñar la función de hash. Aunque existen bibliotecas estándar como SHA-1, la especificación exigía una implementación propia. Tuvimos que investigar técnicas de hashing no criptográficas y decidimos usar una combinación de multiplicación, XOR y rotaciones, similar a la familia de hashes FNV, pero extendida a 128 bits. La validación del hash requirió pruebas con archivos de diferentes tamaños y contenidos para asegurar que colisiones fueran improbables.

Otro problema significativo fue el manejo de la búsqueda distribuida. Coordinar la propagación de mensajes sin causar tormentas de difusión (broadcast storms) requirió implementar correctamente el TTL y el caché de consultas vistas. Al principio, olvidamos excluir al vecino remitente al reenviar, lo que provocaba que los mensajes regresaran al origen. Corregimos esto almacenando el remitente en el mensaje (aunque no se usa explícitamente, el caché evita ciclos) y garantizando que el TTL se decremente correctamente.

La descarga segmentada también presentó retos. Asegurar que cada hilo escriba en la posición correcta del archivo temporal sin sobrescribir datos de otros hilos requirió usar `fseeko()` y abrir el archivo en modo `r+b`. Además, el reintento de rangos fallidos complicó la lógica, ya que un hilo podía fallar y otro podía tomar su rango, pero el archivo temporal ya tenía datos parciales. Finalmente, decidimos que si un rango falla, se intenta con otro par; si ningún par puede completar el rango, la descarga falla por completo.

4.2. Decisiones Únicas

Una decisión importante fue guardar los archivos descargados con el hash como nombre. Esto permite deduplicación automática: si dos archivos tienen el mismo contenido, solo se almacena una copia. Además, facilita la verificación de integridad, ya que el nombre coincide con el hash esperado.

Otra decisión fue implementar el servidor de transferencia y el manejo de mensajes de búsqueda distribuida en el mismo hilo. Esto simplifica el código, ya que ambos usan el mismo socket entrante, y el protocolo distingue entre GET y DSEARCH/DRESULT por el prefijo de la línea.

También optamos por un TTL inicial fijo de 4, pero lo hicimos configurable a través de una constante en `protocol.h`. Esto permite ajustar el alcance de la búsqueda según el tamaño de la red sin modificar el código fuente.

En cuanto a la concurrencia, usamos `pthread_detach()` para los hilos de atención de transferencias, de modo que no tengamos que esperar explícitamente por ellos, evitando fugas de recursos.

5. Aprendizajes Personales

5.1. Aprendizaje de Manfred

Este proyecto me permitió comprender la importancia del diseño de protocolos de comunicación. Siempre me gusta hacer estos proyectos mas que nada porque quedo pensando en que a alguien se le tuvo que ocurrir esto desde cero, y no tenían ni un uno por ciento de las herramientas que tuvimos nosotros al alcance para realizar este proyecto, lo que remarca lo mucho que me queda por aprender. Además, entendí que Capi es un ratón.

5.2. Aprendizaje de David

La parte más desafiante fue la implementación de la búsqueda distribuida. Entender cómo funciona el flooding en redes P2P y aplicarlo en C con hilos me dio una visión práctica de los algoritmos de enrutamiento. También aprendí a depurar problemas de concurrencia usando herramientas como `valgrind` y `gdb` para rastrear condiciones de carrera.

5.3. Aprendizaje de Diego

Me llevo una lección sobre la importancia de la gestión de recursos. Asegurarse de cerrar sockets, liberar memoria y terminar hilos correctamente es crucial para que el sistema no se degrade con el tiempo. Además, el manejo de errores (por ejemplo, cuando un cliente se desconecta abruptamente) debe ser robusto para evitar que el servidor o el cliente fallen.

6. Conclusiones

El proyecto permitió aplicar de manera práctica conceptos fundamentales de sistemas operativos, tales como comunicación mediante sockets, concurrencia, sincronización y sistemas distribuidos. Como resultado, se desarrolló un sistema P2P funcional capaz de registrar, buscar y transferir archivos entre múltiples clientes.

La combinación de un servidor central para el registro de archivos y un mecanismo de búsqueda distribuida proporciona un equilibrio entre simplicidad y tolerancia a fallos. Asimismo, la transferencia segmentada mejora el aprovechamiento de los recursos disponibles y reduce los tiempos de descarga.

La utilización de una función hash propia permitió identificar archivos por contenido y verificar su integridad después de cada transferencia. Por su parte, el uso de TTL y caché de consultas demostró ser una solución efectiva para controlar la propagación de mensajes dentro de la red.

Finalmente, el desarrollo del proyecto fortaleció habilidades relacionadas con programación concurrente, diseño de protocolos y trabajo en equipo, evidenciando la importancia de una arquitectura bien definida en sistemas distribuidos.